

Cryptography in Practice

The Critical 20% That Unlocks 80% of Understanding

A Comprehensive Guide for Security Engineers

Encryption • Hashing • TLS/SSL • Key Management • Common Pitfalls

Introduction: Cryptography for Engineers, Not Mathematicians

As a security engineer, you don't need to understand the mathematics behind elliptic curve cryptography or implement AES from scratch. What you DO need is to understand **how cryptography is used in practice**, when to apply which primitives, how to configure them securely, and—critically—**how they fail when misused**.

This guide focuses on the **practical 20%** of cryptography that enables you to: secure systems correctly, audit configurations for weaknesses, understand why certain attacks work, and make informed decisions when choosing cryptographic solutions.

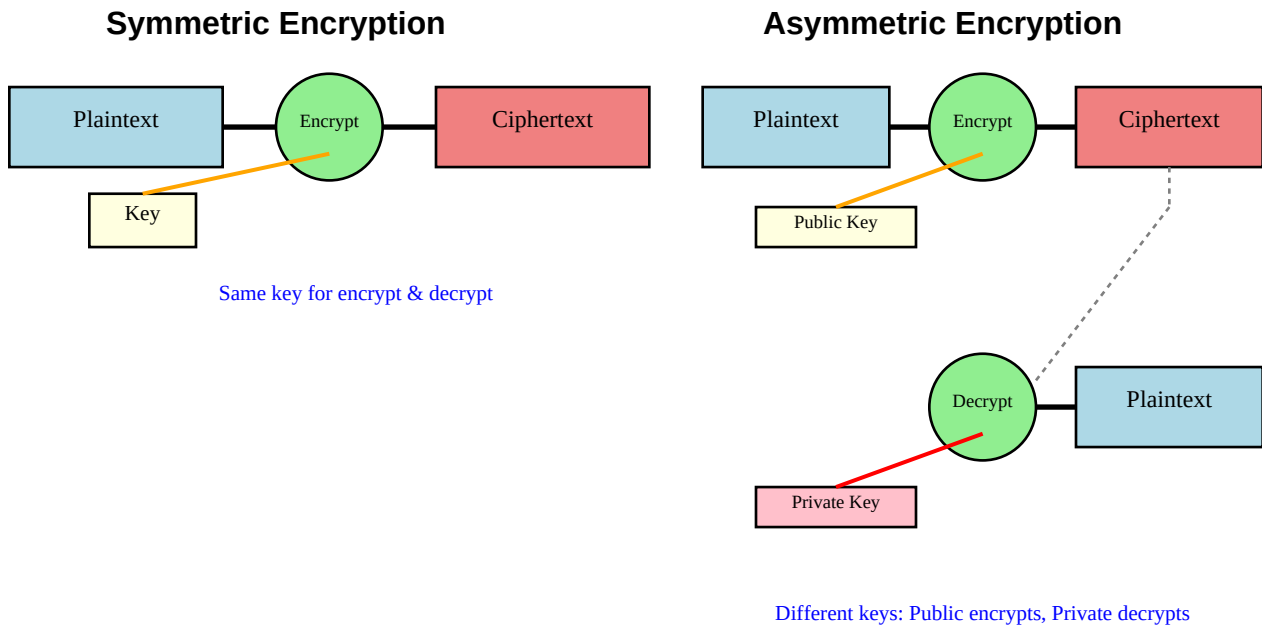
Concept	Why It's Critical for Security Engineers
Encryption: Symmetric vs Asymmetric	Know when to use each; most systems use both in combination
Hashing & MACs	Password storage, integrity verification, authentication; often misused
TLS/SSL in Practice	Securing network communications; misconfigurations are epidemic
Key Management	The weakest link; perfect crypto fails with poor key handling

Critical Rule: Never implement your own cryptography. Use well-vetted libraries and established protocols. Cryptography is easy to get wrong and nearly impossible to verify you got right. Even expert cryptographers use existing implementations.

CONCEPT 1: Symmetric vs Asymmetric Encryption

Two Fundamentally Different Approaches

The Core Distinction: Symmetric encryption uses the same key for encryption and decryption. Asymmetric encryption uses a key pair: public key encrypts, private key decrypts. Each has specific use cases and security properties.



Symmetric Encryption

Symmetric encryption is **fast** and suitable for encrypting large amounts of data. The challenge is key distribution—both parties must securely share the secret key before they can communicate.

Algorithm	Key Size	Block Size	Status	Use Case
AES-256	256 bits	128 bits	RECOMMENDED	General purpose, disk encryption, VPNs
AES-128	128 bits	128 bits	ACCEPTABLE	Performance-critical applications
ChaCha20	256 bits	Stream	RECOMMENDED	Mobile devices, TLS alternative to AES
3DES	168 bits	64 bits	DEPRECATED	Legacy systems only
DES	56 bits	64 bits	INSECURE	Never use
RC4	Variable	Stream	INSECURE	Never use (broken)

Block Cipher Modes

Block ciphers like AES operate on fixed-size blocks (128 bits for AES). To encrypt data larger than one block, you use a **mode of operation**. The mode determines how blocks are chained together:

Mode	Description	Properties	Use Case
GCM	Galois/Counter Mode	Authenticated encryption, parallelizable	RECOMMENDED: TLS, general purpose
CBC	Cipher Block Chaining	Each block depends on previous	Legacy systems, disk encryption
CTR	Counter Mode	Stream-like, parallelizable	Performance-critical
ECB	Electronic Codebook	Each block encrypted independently	NEVER USE: patterns leak
CCM	Counter with CBC-MAC	Authenticated encryption	Resource-constrained devices

Critical: Use **GCM mode** for new implementations. It provides authenticated encryption (confidentiality + integrity). CBC mode requires separate HMAC for integrity. ECB mode is fundamentally flawed—identical plaintext blocks produce identical ciphertext, leaking patterns.

Asymmetric Encryption

Asymmetric encryption is **slow** but solves the key distribution problem. The public key can be freely shared; only the private key must be protected. Typically used to exchange symmetric keys or sign data, not to encrypt large amounts of data.

Algorithm	Key Size	Speed	Status	Use Case
RSA	2048-4096 bits	Slow	ACCEPTABLE	Key exchange, digital signatures
ECC (ECDSA)	256-384 bits	Fast	RECOMMENDED	Mobile, modern systems, Bitcoin
Ed25519	256 bits	Very fast	RECOMMENDED	SSH keys, signing
DH / ECDH	Variable	N/A	RECOMMENDED	Key agreement (not encryption)
RSA < 2048	< 2048 bits	Slow	INSECURE	Never use

Hybrid Encryption: Best of Both Worlds

In practice, systems use **hybrid encryption**: asymmetric crypto to securely exchange a symmetric key, then symmetric crypto to encrypt the actual data. This combines the key distribution benefits of asymmetric with the performance of symmetric.

Example: TLS handshake uses RSA or ECDH to exchange a symmetric key, then uses AES-GCM to encrypt HTTP traffic. The slow asymmetric operation happens once; fast symmetric encryption handles all data transfer.

Real-World Scenario: ECB Mode Penguin

Scenario: A developer encrypts an image using AES-ECB mode. The encrypted file is leaked. Despite being encrypted, the image's content is clearly visible—patterns, shapes, and structure are preserved.

Analysis: ECB mode encrypts each block independently with the same key. Identical plaintext blocks produce identical ciphertext blocks. In images, large areas of the same color (background, sky) create repeating patterns that survive encryption. This is called the "ECB penguin" problem—you can see the outline of the penguin in the encrypted image.

Defense: Never use ECB mode. Use GCM or CBC with IV. These modes ensure identical plaintext blocks produce different ciphertext based on position and randomization (IV/nonce).

Interview Question Pattern

Common question: "When would you use symmetric vs asymmetric encryption?"

Answer:

Symmetric: Encrypting large amounts of data, disk encryption, VPNs, bulk network traffic. Fast, but requires secure key distribution.

Asymmetric: Key exchange, digital signatures, encrypting small data (like symmetric keys). Slow, but public key can be freely shared.

Real world: Almost always use both (hybrid). Asymmetric to establish trust and exchange a symmetric key, symmetric to encrypt actual data. Example: TLS, PGP, SSH all use hybrid encryption.

■ Under-Explored Perspectives: Encryption

1. Initialization Vectors (IVs) and Nonces: CBC and CTR modes require an IV/nonce to prevent pattern leaking. The IV doesn't need to be secret, but it **MUST** be random and unique for each encryption. Reusing IVs with the same key can leak plaintext. This is often misunderstood—developers store IVs as constants.

2. Padding Oracle Attacks: CBC mode requires padding to make data fit block boundaries. If an application leaks information about whether padding is valid (timing differences, error messages), attackers can decrypt data byte-by-byte without the key. This affected major systems (SSL, ASP.NET) but is rarely understood.

3. Key vs Key Encryption Key (KEK): In real systems, you don't encrypt data directly with your master key. You generate a random Data Encryption Key (DEK), encrypt data with DEK, then encrypt DEK with your Key Encryption Key (KEK). This allows key rotation without re-encrypting all data. Most engineers don't know this pattern.

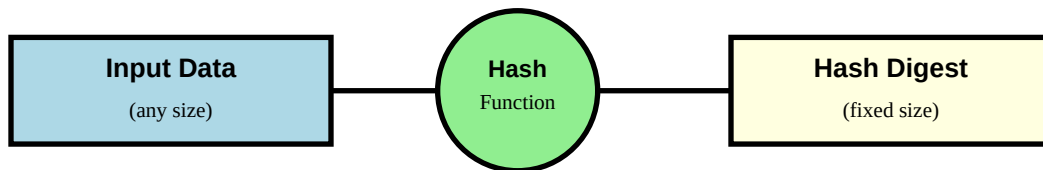
4. Authenticated Encryption is Not Optional: Encryption provides confidentiality, not integrity. An attacker can flip bits in ciphertext to change plaintext without detection. Always use authenticated encryption (GCM, ChaCha20-Poly1305) or Encrypt-then-MAC. Yet many systems use CBC without HMAC.

5. RSA Padding Matters: Raw RSA (textbook RSA) is insecure. Always use padding: OAEP for encryption, PSS for signatures. Older PKCS#1 v1.5 padding has known weaknesses (Bleichenbacher's attack). Yet many legacy systems and tutorials still use it.

CONCEPT 2: Hashing & Message Authentication Codes (MACs)

Ensuring Integrity and Authenticity

Hashing and MACs are NOT encryption—they can't be reversed. Their purpose is **integrity verification** (detect tampering) and **authentication** (prove sender). Understanding when to use which is critical.



Deterministic: same input → same output

One-way: can't reverse hash to get input

Fixed size: always same length output

Avalanche effect: tiny change → completely different hash

Cryptographic Hash Functions

A hash function takes arbitrary-length input and produces a fixed-length output (digest). It must be:

- **Deterministic:** Same input always produces same output
- **One-way:** Computationally infeasible to reverse
- **Collision-resistant:** Extremely hard to find two inputs with same hash
- **Avalanche effect:** Tiny input change = completely different hash

Algorithm	Output Size	Status	Use Case
SHA-256	256 bits	RECOMMENDED	General purpose, blockchain, signatures
SHA-3	224-512 bits	RECOMMENDED	Alternative to SHA-2
BLAKE2	256/512 bits	RECOMMENDED	Faster than SHA-2
SHA-1	160 bits	DEPRECATED	Collision attacks found; use only for legacy
MD5	128 bits	INSECURE	Completely broken; never use for security

Password Hashing: A Special Case

For passwords, you need **slow** hash functions to resist brute-force attacks. General-purpose hashes (SHA-256) are too fast—attackers can test billions of passwords per second. Use password-specific functions:

Algorithm	Type	Status	Key Feature
Argon2	Memory-hard	RECOMMENDED	Winner of password hashing competition, resistant to GPU attacks
bcrypt	CPU-hard	ACCEPTABLE	Configurable work factor, widely supported
scrypt	Memory-hard	ACCEPTABLE	Good GPU resistance
PBKDF2	Iteration-based	ACCEPTABLE	Slower than SHA, but less secure than above
Plain hash	N/A	INSECURE	Trivially cracked with rainbow tables

Always use salt: A random value added to each password before hashing. Without salt, identical passwords have identical hashes, enabling rainbow table attacks. With salt, each password hash is unique even if passwords are identical.

Message Authentication Codes (MACs)

A MAC proves a message came from someone with the secret key and hasn't been tampered with. Unlike a hash, which anyone can compute, a MAC requires the secret key. The most common MAC is **HMAC** (Hash-based MAC).

How HMAC works:

$\text{HMAC}(\text{key}, \text{message}) = \text{hash}(\text{key XOR opad} || \text{hash}(\text{key XOR ipad} || \text{message}))$

Don't worry about the details—just use library implementations. The key point: **HMAC provides authentication and integrity, but not confidentiality**. For both, use authenticated encryption (AES-GCM).

Primitive	Provides	Use Case	Example
Hash	Integrity only	File checksums, non-repudiation	Git commit hashes, blockchain
MAC/HMAC	Integrity + Authentication	Verify sender, detect tampering	JWT signatures, API request signing
Encryption	Confidentiality only	Hide data content	AES-CBC (no integrity!)
Authenticated Encryption	All three	Modern secure communication	AES-GCM, ChaCha20-Poly1305

Real-World Scenario: MD5 Collision Attack

Scenario: An attacker creates two different executable files with identical MD5 hashes. One is benign software; the other is malware. They submit the benign version for code signing. After it's signed, they distribute the malware version with the same signature (since MD5 hashes match).

Analysis: MD5 is cryptographically broken—collisions can be generated in seconds. Digital signatures rely on hash uniqueness. If two files have the same hash, they have the same signature. This attack worked against real certificate authorities.

Defense: Never use MD5 or SHA-1 for security. Use SHA-256 or better. Modern certificate authorities now require SHA-256 minimum.

Interview Question Pattern

Common question: "Why can't you use a regular hash function like SHA-256 for password storage?"

Answer: SHA-256 is too fast. Modern GPUs can compute billions of SHA-256 hashes per second. An attacker with a stolen password database can try every word in the dictionary, common passwords, and variations in seconds to minutes. Password hashing functions like Argon2 or bcrypt are intentionally slow (configurable work factor) and memory-hard, making brute-force attacks impractical. Plus, they handle salting automatically. SHA-256 without salt + iteration is basically plaintext for an attacker.

■ Under-Explored Perspectives: Hashing & MACs

1. Length Extension Attacks: SHA-1, SHA-256, MD5 are vulnerable to length extension attacks if used directly as MACs. Given `hash(secret || message)`, an attacker can compute `hash(secret || message || attacker_data)` without knowing the secret. This is why HMAC exists—it's immune to this attack. Yet developers still use `SHA256(secret || message)` as a MAC.

2. Timing Attacks on Hash Comparison: Comparing hashes byte-by-byte in a loop exits early on mismatch. This timing difference leaks information. Use constant-time comparison functions. In Python: `hmac.compare_digest()`. In Node: `crypto.timingSafeEqual()`. Regular `'=='` is vulnerable.

3. Rainbow Tables Still Work: Pre-computed hash databases for common passwords. Salt prevents rainbow tables, but if your salt is static (same for all users) or predictable, attackers can pre-compute for that salt. Salt must be random and unique per user.

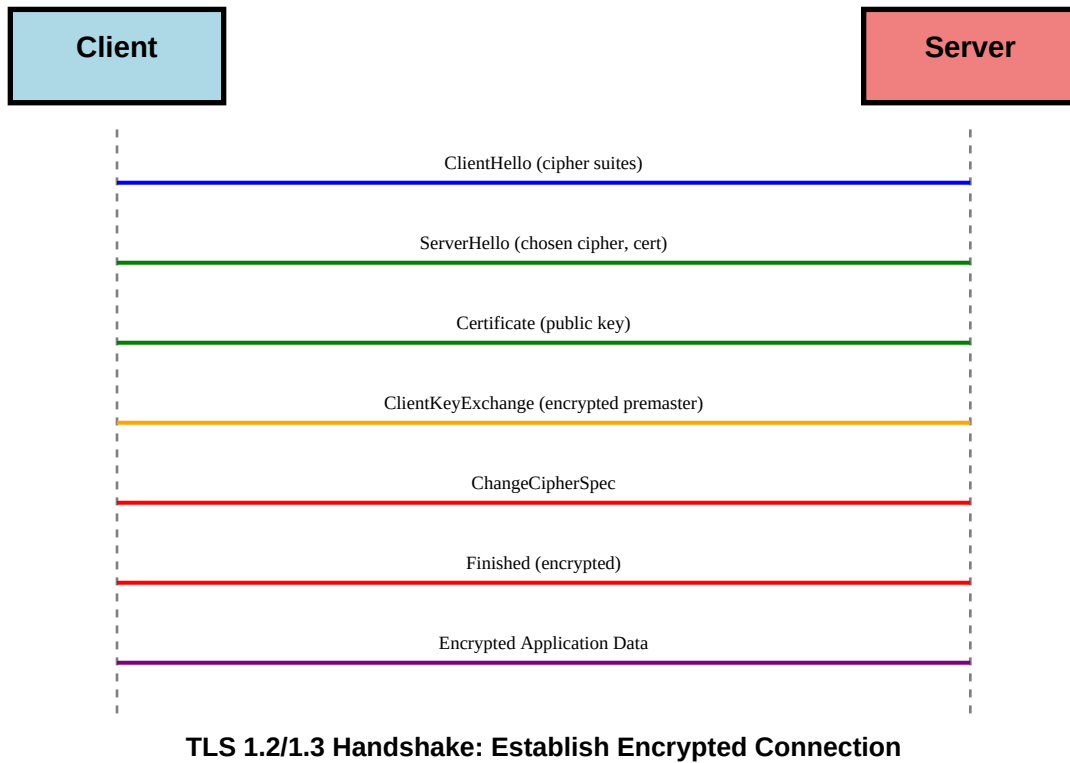
4. The bcrypt Truncation Issue: bcrypt only uses first 72 bytes of input. If passwords are longer, they're truncated, reducing entropy. Best practice: hash password with SHA-256 first, then bcrypt the hash. This also handles Unicode correctly.

5. Collision vs Preimage Resistance: Collision = two different inputs with same hash. Preimage = given hash, find any input that produces it. For digital signatures, collision resistance matters (attacker controls both inputs). For password cracking, preimage resistance matters (attacker has the hash, needs the password). Different threat models, but often confused.

CONCEPT 3: TLS/SSL in Practice

Securing Network Communications

TLS (Transport Layer Security) is the protocol that secures HTTPS, protects email (STARTTLS), and encrypts VPNs. Understanding how TLS works and how it fails is essential for securing web applications and troubleshooting production issues.



TLS Handshake: Establishing Secure Connection

The TLS handshake establishes:

1. **Identity:** Server proves identity with certificate
2. **Agreement:** Both sides agree on encryption algorithms
3. **Key exchange:** Securely establish symmetric encryption key
4. **Integrity:** Both sides can detect tampering

Certificate Validation: The Critical Step

The server's certificate proves its identity. Proper validation checks:

Check	Purpose	Failure Impact
-------	---------	----------------

Signature	Certificate signed by trusted CA	Self-signed cert accepted = MITM possible
Hostname	Certificate matches requested domain	Wrong domain cert accepted = MITM possible
Validity period	Certificate not expired	Expired cert may indicate compromise
Revocation	Certificate not revoked (CRL/OCSP)	Compromised certs may still be accepted
Chain of trust	Valid path to root CA	Broken chain = untrustworthy cert

TLS Versions and Cipher Suites

Version	Status	Key Features	Notes
TLS 1.3	RECOMMENDED	1-RTT handshake, modern ciphers only	Fastest, most secure
TLS 1.2	ACCEPTABLE	Widely supported, but allows weak ciphers	Still common, configure carefully
TLS 1.1	DEPRECATED	Legacy support	Disable in new systems
TLS 1.0	INSECURE	Vulnerable (BEAST, POODLE)	Disable immediately
SSL 3.0	INSECURE	Completely broken	Never use

Cipher suite example: TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384

- **ECDHE:** Key exchange algorithm (Elliptic Curve Diffie-Hellman Ephemeral)
- **RSA:** Authentication algorithm
- **AES_256_GCM:** Symmetric encryption (256-bit AES in GCM mode)
- **SHA384:** Hash function for integrity

Common TLS Misconfigurations

Misconfiguration	Impact	Detection	Fix
Weak cipher suites	Vulnerable to cryptanalysis	SSL Labs scan	Disable weak ciphers in server config
Missing HSTS header	Downgrade attacks possible	Security headers check	Add Strict-Transport-Security header
Self-signed certificate	MITM attacks possible	Browser warning	Get certificate from trusted CA
Certificate for wrong domain	MITM attacks possible	Hostname mismatch error	Use correct cert or wildcard/SAN cert
Disabled cert validation	All attacks possible	Code review, penetration test	Never disable validation in production

Real-World Scenario: Certificate Validation Disabled

Scenario: A developer encounters certificate errors in development (self-signed cert). To "fix" it quickly, they add:

```
curl_setopt($ch, CURLOPT_SSL_VERIFYPEER, false);
```

This code ships to production. An attacker performs MITM attack, intercepts all HTTPS traffic.

Analysis: Disabling certificate validation removes all TLS security. The connection is still encrypted, but you're encrypting to the attacker's server, not the real server. This is one of the most common and dangerous misconfigurations.

Defense: NEVER disable certificate validation in production. If you must in development, use environment-specific configuration, conditional logic, and loud warnings. Better: use local CA

for dev certificates.

Interview Question Pattern

Common question: "What is perfect forward secrecy and why does it matter?"

Answer: Perfect Forward Secrecy (PFS) ensures that if a server's private key is compromised, past communications remain secure. Without PFS, an attacker who steals the server's RSA private key can decrypt all previously captured traffic. With PFS (using ECDHE or DHE key exchange), each session uses temporary keys that are discarded after use. Even if the server's key is stolen later, old sessions can't be decrypted. PFS is now standard in TLS 1.3 and should be enforced in TLS 1.2 by prioritizing ECDHE cipher suites.

■ Under-Explored Perspectives: TLS/SSL

1. OCSP Stapling vs. Regular OCSP: Certificate revocation can be checked via OCSP, but this creates privacy issues (CA sees every site you visit) and adds latency. OCSP stapling solves this—the server includes the OCSP response in the handshake. It's more private and faster, but must be configured server-side. Most servers don't enable it.

2. The NULL Cipher Suite: TLS has a 'NULL' cipher suite that provides no encryption at all. It's intended for debugging but if enabled on production servers, allows connections with zero encryption while appearing to use HTTPS. Always explicitly disable NULL ciphers in server configuration.

3. Session Resumption Vulnerabilities: TLS session resumption reuses negotiated parameters to avoid full handshakes. Session IDs and session tickets enable this, but if implemented incorrectly, they can leak information or allow replay attacks. Session ticket keys must be rotated and not shared across all servers.

4. SNI Leakage: Server Name Indication (SNI) reveals the hostname in plaintext during TLS handshake (before encryption). An eavesdropper knows which site you're visiting even though content is encrypted. Encrypted SNI (ESNI/ECH) solves this but isn't widely deployed. This is a privacy issue often overlooked.

5. Certificate Transparency is Not Verification: Certificate Transparency (CT) logs all issued certificates publicly, helping detect misissued certs. But CT logging doesn't prevent misissued certs—it only makes them discoverable after the fact. It's a detective control, not preventive. Many assume CT prevents all certificate attacks.

CONCEPT 4: Key Management - The Weakest Link

Perfect Crypto Fails with Poor Key Handling

You can use AES-256-GCM with perfect implementation, but if the encryption key is stored in plaintext in your source code, you have zero security. **Key management is where most crypto failures occur in practice.** Keys must be generated securely, stored securely, used securely, rotated regularly, and destroyed completely when no longer needed.

Key Generation

Keys must be generated using cryptographically secure random number generators (CSPRNGs), not regular random functions.

Language	INSECURE (Don't Use)	SECURE (Use This)
Python	random.random()	secrets module, os.urandom()
JavaScript	Math.random()	crypto.getRandomValues()
Java	java.util.Random	java.security.SecureRandom
C/C++	rand()	/dev/urandom, CryptGenRandom()
PHP	rand(), mt_rand()	random_bytes()

Key Storage: Hierarchy of Security

Method	Security Level	Use Case	Pros/Cons
Hardware Security Module (HSM)	HIGH	Root CA keys, payment processing	Tamper-resistant, expensive, complex
Trusted Platform Module (TPM)	HIGH	Disk encryption, device identity	Hardware-backed, device-specific
Key Management Service (KMS)	HIGH	Cloud encryption, secrets mgmt	Managed service, audit logs, permissions
Encrypted key file	MEDIUM	Application secrets	Protected by master key, still on disk
Environment variables	LOW	Development only	Not encrypted, visible to all processes
Configuration file	VERY LOW	Never use in production	Plaintext, often in version control
Hardcoded in source	INSECURE	Never	Worst possible practice

Key Rotation

Keys should be rotated regularly to limit exposure if compromised. Rotation frequency depends on risk:

High-risk keys: API tokens, database encryption keys → Every 30-90 days

Medium-risk: Application secrets, service account keys → Every 6-12 months

Low-risk: Certificate private keys (with short-lived certs) → Certificate lifetime

Immediate rotation required: Any suspected compromise, employee departure with key access

Envelope encryption pattern: Don't encrypt data directly with your master key. Generate a Data Encryption Key (DEK), encrypt data with DEK, then encrypt DEK with your Key Encryption Key (KEK). To rotate keys, you only re-encrypt the small DEKs, not all data.

Real-World Scenario: AWS Access Key in GitHub

Scenario: A developer accidentally commits AWS access keys to a public GitHub repository. Within minutes, automated bots discover the keys, use them to spin up expensive EC2 instances for cryptocurrency mining, and rack up \$50,000 in charges in 24 hours.

Analysis: Bots constantly scan GitHub for committed secrets. Once found, keys are immediately exploited. The issue isn't GitHub—it's storing secrets in source code. Even private repositories are vulnerable (insider threats, repository access changes).

Defense:

1. Never commit secrets to version control—use .gitignore
2. Use secrets management solutions (AWS Secrets Manager, HashiCorp Vault)
3. Enable pre-commit hooks to scan for secrets (git-secrets, trufflehog)
4. Rotate compromised keys immediately
5. Monitor for unexpected AWS resource creation

Interview Question Pattern

Common question: "How would you securely manage encryption keys for a multi-tier application?"

Approach:

1. Use cloud KMS (AWS KMS, Azure Key Vault) or dedicated secrets manager
2. Implement envelope encryption: Master key in KMS, DEKs encrypted by master key
3. Application retrieves DEKs at startup, decrypts with KMS API
4. Never log or persist decrypted keys; keep in memory only
5. Rotate DEKs quarterly, master key annually
6. Enforce least privilege: Each microservice gets only its keys
7. Enable audit logging: Log all key access and usage
8. Secure key destruction when decommissioning systems

■ Under-Explored Perspectives: Key Management

1. Key Deletion is Hard: Securely deleting keys is surprisingly difficult. On SSDs and cloud storage, deleted data may persist. Best practice: before deletion, overwrite the key with random data, then delete. For HSMs, cryptographic erasure (destroy the key encrypting the keys) is used. Simply hitting 'delete' isn't sufficient for sensitive keys.

2. Key Ceremony: For extremely sensitive keys (root CA keys, cryptocurrency signing keys), a formal 'key ceremony' is performed. Multiple trusted individuals participate, each provides a key share (Shamir's Secret Sharing), and the entire process is audited and recorded. Most engineers never encounter this but it's standard practice for high-value keys.

3. Key Escrow vs. Key Recovery: Key escrow (giving a copy to a third party) enables law enforcement access or data recovery but creates a honeypot. Key recovery (splitting keys across multiple parties) achieves similar goals without single point of failure. The tradeoffs are complex and often misunderstood.

4. Weak Key Derivation: Deriving encryption keys from passwords is common but dangerous if done wrong. Use proper KDFs (PBKDF2, Argon2) with high iteration counts. Simply hashing a password with SHA-256 creates a weak key vulnerable to brute force. Many applications use weak KDFs, making encryption essentially worthless.

5. The M-of-N Problem: For critical operations, you might require M out of N keyholders to authorize (e.g., 3 of 5 admins must approve). This is implemented with threshold cryptography or multi-signature schemes. It's powerful for security but adds operational complexity. Backup and recovery procedures must account for this.

Conclusion: Practical Cryptography Mastery

Cryptography is the mathematical foundation of security, but as a security engineer, your focus is on **correct application**, not inventing new algorithms. The vast majority of cryptographic failures come from misuse of strong primitives, not from breaking the math.

The four concepts covered—**symmetric vs asymmetric encryption, hashing and MACs, TLS/SSL in practice, and key management**—form the practical toolkit every security engineer needs. Use strong algorithms (AES-GCM, SHA-256, TLS 1.3). Configure them correctly (no ECB, no disabled cert validation). Manage keys properly (KMS, rotation, never in source code). Audit everything.

Key Takeaways

1. Use authenticated encryption (AES-GCM) for confidentiality + integrity. Never use ECB mode. Always use random IVs.
2. Hash functions (SHA-256) for integrity. Password hashing functions (Argon2, bcrypt) for passwords with salt. Never use MD5/SHA-1.
3. TLS 1.3 or 1.2 with strong cipher suites. Always validate certificates. Never disable verification in production.
4. Key management is where crypto fails in practice. Use KMS/secrets manager, rotate regularly, never hardcode or commit to Git.
5. Never implement your own crypto. Use vetted libraries. Most failures come from misuse of strong crypto, not weak algorithms.

Your path forward: The best way to learn cryptography in practice is to break it. Study common vulnerabilities (Padding Oracle, timing attacks, weak randomness). Set up intentionally vulnerable apps (WebGoat, DVWA) and exploit crypto weaknesses. Understanding how crypto fails makes you far better at using it correctly.